



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea Magistrale in Informatica

Metodi del Calcolo Scientifico: Assignment 1

Gruppo:

Broglia Matteo – 899562

Caputo Lorenzo – 894528

Giuggioli Daniel – 894415

Anno Accademico 2025 - 2026

Indice

1	Struttura Libreria	1
1.1	Architettura	1
1.1.1	Organizzazione File	1
1.2	Esecuzione	2
2	Metodi Iterativi	3
2.1	Metodo di Jacobi	3
2.2	Metodo di Gauss-Seidel	4
2.3	Metodo del Gradiente	4
2.4	Metodo del Gradiente Coniugato	5
3	Risultati	7
3.1	Parametri di Test	7
3.2	Analisi dei Risultati	10
3.2.1	Iterazioni vs Tolleranza	10
3.2.2	Tempo vs Tolleranza	11
3.2.3	Errore vs Tolleranza	12
3.2.4	Confronto per Famiglia di Matrici	13
3.3	Analisi Complessiva	16
3.3.1	Tabelle Riassuntive	17
3.4	Processo di Ottimizzazione	18
3.5	Accelerazione con Numba	18
4	Conclusioni	21
4.1	Valutazione dei Metodi Iterativi	21
4.2	Impatto dell'Ottimizzazione Numerica e del JIT	22

Capitolo 1

Struttura Libreria

Questo progetto implementa una libreria Python per la risoluzione numerica di sistemi lineari $Ax = b$ con matrice A simmetrica e definita positiva, mediante metodi iterativi classici. La libreria è organizzata in modo modulare, con una chiara separazione tra l'implementazione dei solutori, la gestione dell'I/O e gli script di validazione e visualizzazione dei risultati.

Il codice è reperibile tramite GitHub al seguente indirizzo

<https://github.com/mbroglio/LinearSolvers>

1.1 Architettura

Ogni metodo iterativo è implementato come sottoclasse che specializza il metodo `solve(A, b)`, restituendo un oggetto `SolverResult` contenente:

- Soluzione approssimata x
- Numero di iterazioni
- Tempo di esecuzione
- Residuo relativo finale
- Flag di convergenza

1.1.1 Organizzazione File

- **linear_solvers/**: Package principale organizzato in due sottomoduli (implementazione standard e ottimizzata con Numba)
 - **baseline/**: Insieme dei moduli con l'implementazione base dei solutori:
 - * `base.py`: Classe astratta `IterativeSolver` che definisce l'interfaccia comune e metodi condivisi
 - * `iterative_methods.py`: Implementazioni concrete dei metodi iterativi
 - **numba/**: Insieme dei moduli con l'implementazione ottimizzata tramite il compilatore JIT Numba (stessa struttura in file della baseline)

- **data/**: Matrici di test in formato Matrix Market (`spa1.mtx`, `spa2.mtx`, `vem1.mtx`, `vem2.mtx`)
- **main.py**: Entry point da riga di comando per validare i metodi iterativi su una matrice `.mtx`, configurando tolleranza e numero massimo di iterazioni

Al fine di testare tutte le matrici con diverse tolleranze e generare i rispettivi grafici vengono inclusi, all'interno della directory **runner**, i seguenti file o cartelle:

- **run_baseline.py**: Script per eseguire i test con l'implementazione di base
- **run_numba.py**: Script per eseguire i test con l'implementazione ottimizzata Numba
- **results/**: Tabelle riassuntive, report dettagliati in CSV e grafici dei risultati ottenuti
- **plot_results.py**: Script per la generazione di grafici comparativi
- **generate_plots.py**: Script per generare i pattern di sparsità

1.2 Esecuzione

Lo script `main.py` fornisce un'interfaccia a riga di comando per eseguire in sequenza tutti i solutori implementati e confrontarne le prestazioni sulla stessa istanza del problema lineare.

Sintassi

```
python main.py <matrice.mtx> [--tol TOLLERANZA] [--max-iter N]
```

Parametri

- **matrix** (posizionale, obbligatorio): percorso al file Matrix Market `.mtx`
- **--tol** (opzionale, default `1e-10`): tolleranza per il criterio di arresto
- **--max-iter** (opzionale, default `20000`): numero massimo di iterazioni consentite. La libreria impone a runtime che questo valore sia ≥ 20000 , come da specifica; valori inferiori sollevano un'eccezione

Esempio

```
python main.py data/spa1.mtx --tol 1e-8 --max-iter 20000
```

Gestione Errori

Se un solutore solleva un'eccezione (ad esempio per mancata convergenza o preconditioni non soddisfatte), `main.py` non interrompe l'intera esecuzione: segnala l'errore nella riga corrispondente della tabella e prosegue con i metodi successivi.

Capitolo 2

Metodi Iterativi

2.1 Metodo di Jacobi

Metodo basato sulla decomposizione $A = D - L - U$ (con L triangolare inferiore e U triangolare superiore, entrambe invertite di segno). A ogni passo calcola $\vec{x}^{(k+1)}$ usando solo i valori dell'iterata precedente $\vec{x}^{(k)}$, quindi tutti gli aggiornamenti sono indipendenti tra loro. La convergenza è garantita se la matrice è a dominanza diagonale stretta oppure se è simmetrica e definita positiva.

Dato il sistema $A\vec{x} = \vec{b}$ con $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$:

1. **Input:** Matrice A , Vettore $\vec{b}$, Tolleranza ϵ , Iterazioni Massime k_{max}
2. **Inizializzazione:** $\vec{x}^{(0)} = 0$
3. **Precondizione:** Verificare che $a_{ii} \neq 0$ per $i = 1, \dots, n$
4. **Iterazione:** Per $k = 0, 1, 2, \dots$ fino a convergenza:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right) \quad (2.1)$$

5. **Criterio di Arresto:** $\frac{\|A\vec{x}^{(k)} - \vec{b}\|}{\|\vec{b}\|} < \epsilon$ oppure $k \geq k_{max}$

2.2 Metodo di Gauss-Seidel

Viene usata la stessa decomposizione di Jacobi ma, a differenza di quest'ultimo, riutilizza immediatamente i valori già aggiornati all'interno della stessa iterazione. In forma compatta si può scrivere $(D+L)\vec{x}^{(k+1)} = \vec{b} - U\vec{x}^{(k)}$. Tale scelta rende in genere la convergenza più rapida rispetto a Jacobi; la convergenza è garantita se la matrice è a dominanza diagonale stretta oppure se è simmetrica e definita positiva.

Dato il sistema $A\vec{x} = \vec{b}$ con $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$:

1. **Input:** Matrice A , Vettore $\vec{b}$, Tolleranza ϵ , Iterazioni Massime k_{max}
2. **Inizializzazione:** $\vec{x}^{(0)} = 0$
3. **Precondizione:** Verificare che $a_{ii} \neq 0$ per $i = 1, \dots, n$
4. **Iterazione:** Per $k = 0, 1, 2, \dots$ fino a convergenza:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) \quad (2.2)$$

5. **Criterio di Arresto:** $\frac{\|A\vec{x}^{(k)} - \vec{b}\|}{\|\vec{b}\|} < \epsilon$ oppure $k \geq k_{max}$

2.3 Metodo del Gradiente

Il metodo del Gradiente (steepest descent) è un metodo non stazionario per matrici simmetriche definite positive (SPD). Interpreta la soluzione di $A\vec{x} = \vec{b}$ come minimizzazione della funzione quadratica associata e, a ogni iterazione, sceglie come direzione il residuo $\vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)}$, con passo ottimale lungo tale direzione. La convergenza del metodo è garantita solo in presenza di una matrice SPD.

Dato il sistema $A\vec{x} = \vec{b}$ con $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$:

1. **Input:** Matrice A SPD, Vettore $\vec{b}$, Tolleranza ϵ , Iterazioni Massime k_{max}
2. **Precondizione:** Verificare che $A = A^T$ (simmetria)
3. **Inizializzazione:** $\vec{x}^{(0)} = 0$, $\vec{r}^{(0)} = \vec{b} - A\vec{x}^{(0)} = \vec{b}$
4. **Iterazione:** Per $k = 0, 1, 2, \dots$ fino a convergenza:

$$\alpha_k = \frac{(\vec{r}^{(k)})^T \vec{r}^{(k)}}{(\vec{r}^{(k)})^T A \vec{r}^{(k)}} \quad (2.3)$$

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \alpha_k \vec{r}^{(k)} \quad (2.4)$$

$$\vec{r}^{(k+1)} = \vec{r}^{(k)} - \alpha_k A \vec{r}^{(k)} \quad (2.5)$$

5. **Controllo:** Se $(\vec{r}^{(k)})^T A \vec{r}^{(k)} \leq 0$, la matrice non è definita positiva
6. **Criterio di Arresto:** $\frac{\|A\vec{x}^{(k)} - \vec{b}\|}{\|\vec{b}\|} < \epsilon$ oppure $k \geq k_{max}$

2.4 Metodo del Gradiente Coniugato

Il Gradiente Coniugato è un'estensione del metodo del Gradiente che utilizza direzioni coniugate rispetto alla matrice A . In teoria converge in al più n iterazioni per matrici SPD; tuttavia, nella pratica, a causa dell'aritmetica floating-point (precisione finita della macchina), si verifica una perdita di ortogonalità tra le direzioni coniugate, che può ritardare o anticipare la convergenza effettiva.

Dato il sistema $A\vec{x} = \vec{b}$ con $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$:

1. **Input:** Matrice A SPD, Vettore $\vec{b}$, Tolleranza ϵ , Iterazioni Massime k_{max}
2. **Precondizione:** Verificare che $A = A^T$ (simmetria)
3. **Inizializzazione:** $\vec{x}^{(0)} = 0$, $\vec{r}^{(0)} = \vec{b}$, $\vec{p}^{(0)} = \vec{r}^{(0)}$
4. **Iterazione:** Per $k = 0, 1, 2, \dots$ fino a convergenza:

$$\alpha_k = \frac{(\vec{r}^{(k)})^T \vec{r}^{(k)}}{(\vec{p}^{(k)})^T A \vec{p}^{(k)}} \quad (2.6)$$

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \alpha_k \vec{p}^{(k)} \quad (2.7)$$

$$\vec{r}^{(k+1)} = \vec{r}^{(k)} - \alpha_k A \vec{p}^{(k)} \quad (2.8)$$

$$\beta_k = \frac{(\vec{r}^{(k+1)})^T \vec{r}^{(k+1)}}{(\vec{r}^{(k)})^T \vec{r}^{(k)}} \quad (2.9)$$

$$\vec{p}^{(k+1)} = \vec{r}^{(k+1)} + \beta_k \vec{p}^{(k)} \quad (2.10)$$

5. **Controllo:** Se $(\vec{p}^{(k)})^T A \vec{p}^{(k)} \leq 0$, la matrice non è definita positiva

6. **Criterio di Arresto:** $\frac{\|A\vec{x}^{(k)} - \vec{b}\|}{\|\vec{b}\|} < \epsilon$ oppure $k \geq k_{max}$

Capitolo 3

Risultati

3.1 Parametri di Test

La validazione è stata eseguita su quattro matrici sparse simmetriche e definite positive:

- **spa1.mtx**: Dimensione $n = 4363$
- **spa2.mtx**: Dimensione $n = 11908$
- **vem1.mtx**: Dimensione $n = 2663$
- **vem2.mtx**: Dimensione $n = 5093$

Per ogni matrice A :

- Soluzione Esatta: $x_{\text{exact}} = [1, 1, \dots, 1]^T$
- Termine Noto: $b = A \cdot x_{\text{exact}}$
- Tolleranze Testate: $\{10^{-10}, 10^{-8}, 10^{-6}, 10^{-4}\}$
- Iterazioni Massime: $k_{\text{max}} = 20000$

Hardware

I test sono stati eseguiti su un sistema con le seguenti specifiche:

- Processore: Intel Core i5-12600KF (10 core, 3.7 GHz)
- RAM: 32 GB DDR4
- Scheda Grafica: AMD Radeon RX 6800 XT (16 GB GDDR6)
- Sistema Operativo: Windows 11
- Librerie: NumPy, SciPy, Matplotlib (Generazione Grafici)

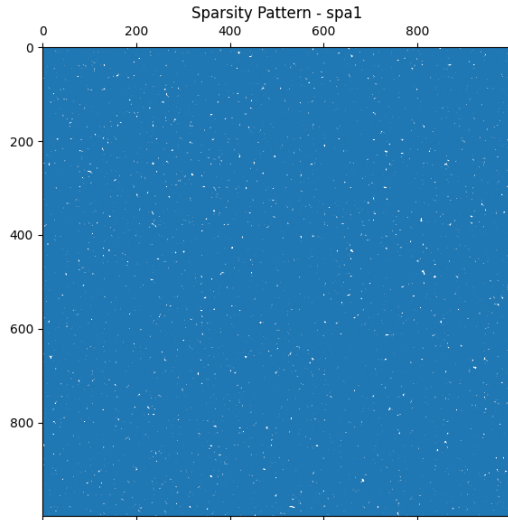


Figura 3.1: Pattern di Sparsità - spa1

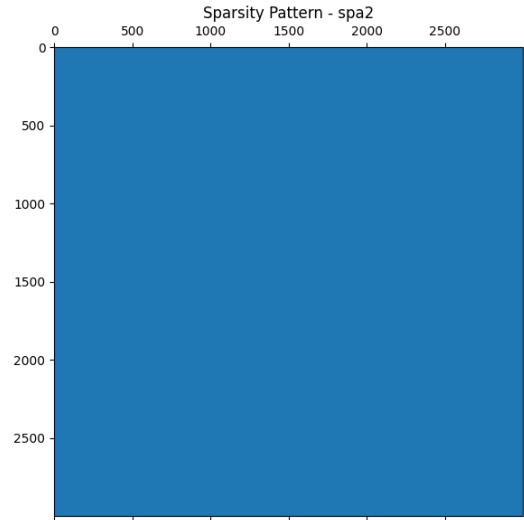


Figura 3.2: Pattern di Sparsità - spa2

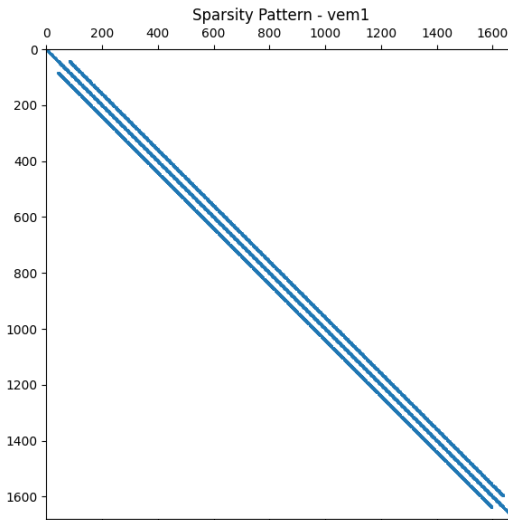


Figura 3.3: Pattern di Sparsità - vem1

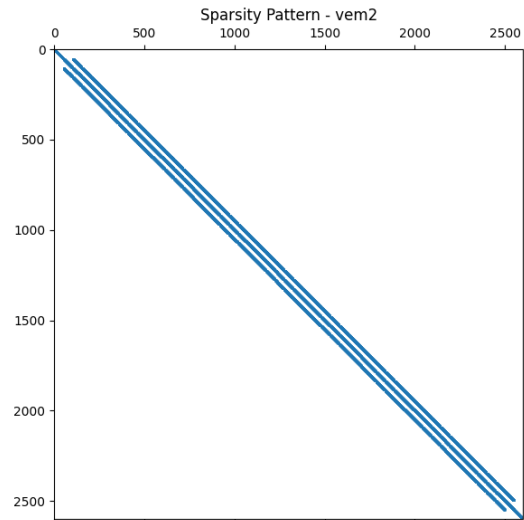


Figura 3.4: Pattern di Sparsità - vem2

Rappresentazione in Memoria delle Matrici

Per garantire la massima efficienza degli algoritmi, tutte le matrici sono state importate e trattate utilizzando il formato compresso CSR (Compressed Sparse Row) tramite l'impiego del metodo `.tocsr()` di SciPy. L'adozione del formato sparso rispetto ad una rappresentazione densa (`.toarray()`) è dovuta a:

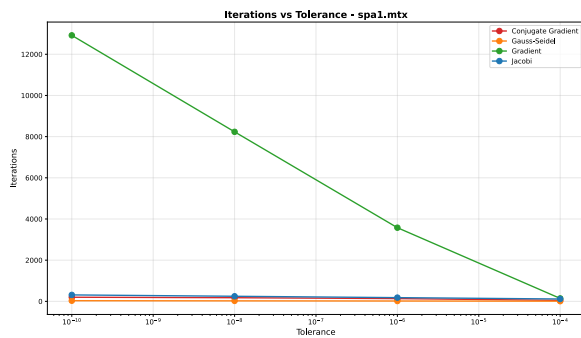
- **Costo Spaziale (Memoria):** Matrici come `spa2.mtx` raggiungono dimensioni $n \approx 12000$, rivelandosi formate per la quasi totalità da zeri. Un formato denso allocherebbe in memoria N^2 elementi, causando un consumo eccessivo e ingiustificato di RAM. Il formato CSR memorizza invece esclusivamente i valori non nulli e i loro relativi indici testuali
- **Costo Temporale (Complessità):** Utilizzare un formato denso porterebbe ad eseguire il prodotto matrice-vettore con una complessità di $O(N^2)$. Il formato CSR

riduce questo costo a $O(NNZ)$ (dove NNZ è il numero di elementi non nulli), alleggerendo notevolmente i calcoli per tutte le iterazioni

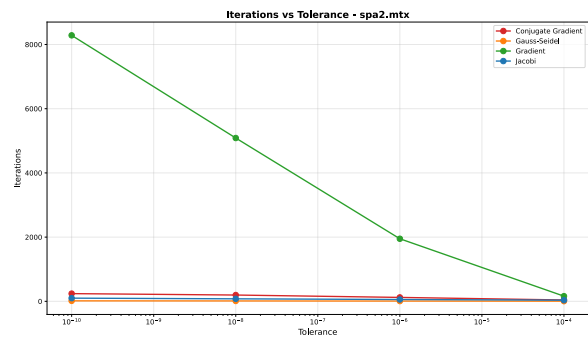
3.2 Analisi dei Risultati

Tutti i metodi convergono su tutte e quattro le matrici testate, per ogni livello di tolleranza considerato. La convergenza garantita in ogni scenario consente di concentrare l'analisi sull'efficienza computazionale, le differenze più marcate riguardano la quantità di iterazioni necessarie e il costo temporale associato.

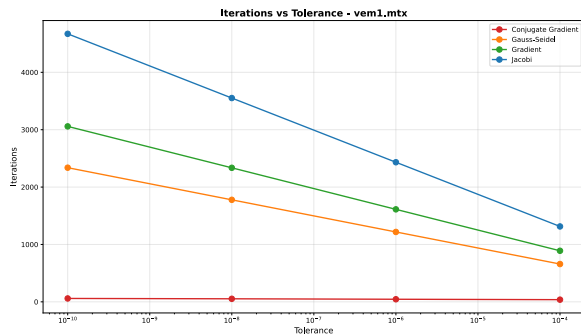
3.2.1 Iterazioni vs Tolleranza



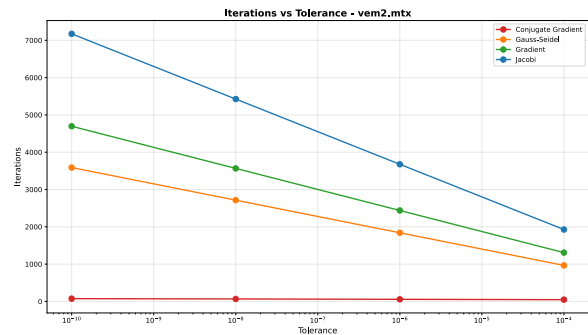
(a) spa1



(b) spa2



(c) vem1



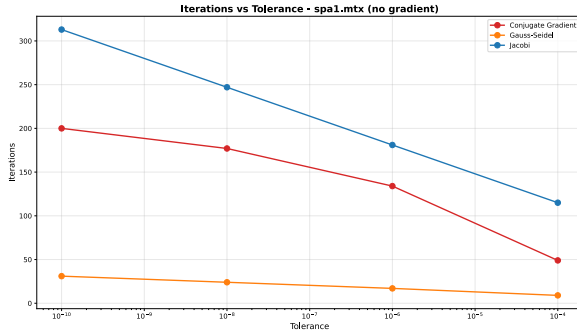
(d) vem2

In tutti e quattro i grafici si osserva un andamento crescente, questo poiché minore è la tolleranza, maggiore sarà il numero di iterazioni al fine della convergenza. Il comportamento varia però sensibilmente in funzione della struttura della matrice.

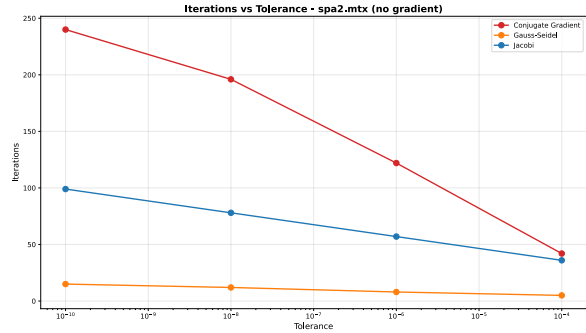
Sulle matrici della famiglia **vem**, il Gradiente Coniugato risulta essere nettamente migliore, richiedendo solo 74 iterazioni su **vem2** (tolleranza 10^{-10}), contro le 3589 di Gauss-Seidel e le 7174 di Jacobi.

Al contrario, sulle matrici **spa**, Gauss-Seidel risulta essere il metodo con il minor numero di iterazioni: su **spa2** (a 10^{-10}) converge in sole 15 iterazioni, superando ampiamente il Gradiente Coniugato che ne richiede 240. Il Gradiente semplice e Jacobi si confermano mediamente i metodi che richiedono un maggior numero di iterazioni al fine della convergenza.

spa1 e spa2 senza Gradiente



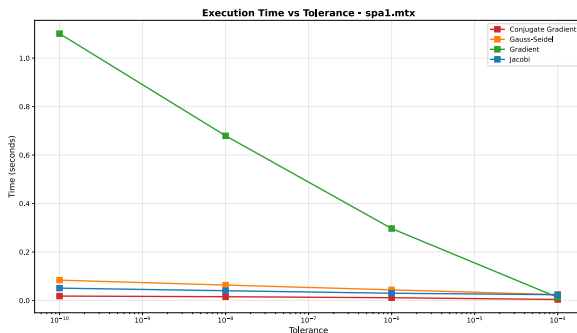
(a) spa1 (senza Gradiente)



(b) spa2 (senza Gradiente)

Limitando il confronto a Jacobi, Gauss-Seidel e Gradiente Coniugato, si nota meglio il comportamento anomalo sulle matrici **spa**. Gauss-Seidel è particolarmente efficiente, mantenendo il conteggio delle iterazioni sotto le 35 per ogni livello di tolleranza su **spa1** e non oltre le 15 su **spa2**.

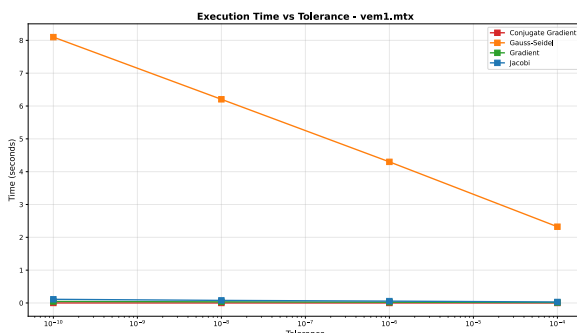
3.2.2 Tempo vs Tolleranza



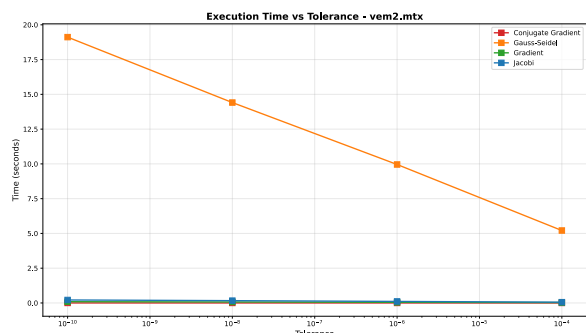
(a) spa1



(b) spa2



(c) vem1



(d) vem2

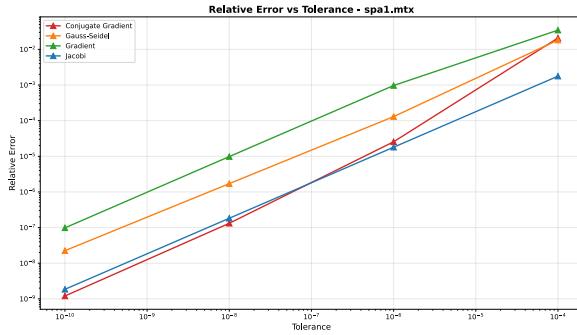
L'analisi dei tempi di esecuzione mostra risultati controintuitivi rispetto al numero delle iterazioni.

Su **spa1** a 10^{-10} , nonostante Gauss-Seidel esegua meno iterazioni (31 contro le 200 del Gradiente Coniugato), il Gradiente Coniugato risulta nettamente più veloce (0.018s contro 0.084s), suggerendo un minor costo computazionale per iterazione.

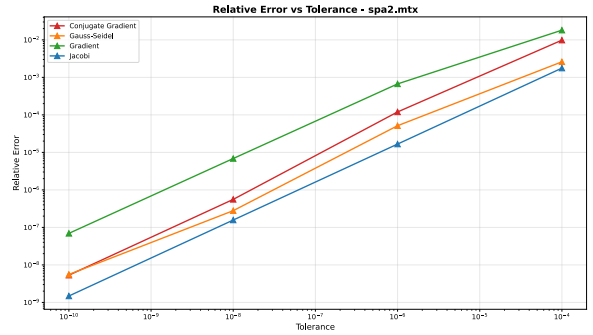
Su **spa2**, invece, la rapidità di convergenza di Gauss-Seidel lo rende il metodo assoluto più veloce (0.17s contro 0.25s del Gradiente Coniugato).

Sulle matrici **vem**, i tempi dei metodi stazionari risultano crescere notevolmente. Su **vem2**, con precisione 10^{-10} , Gauss-Seidel impiega ben ~ 19 s, Jacobi richiede ~ 0.21 s, mentre il Gradiente Coniugato risolve il sistema in appena 0.002s.

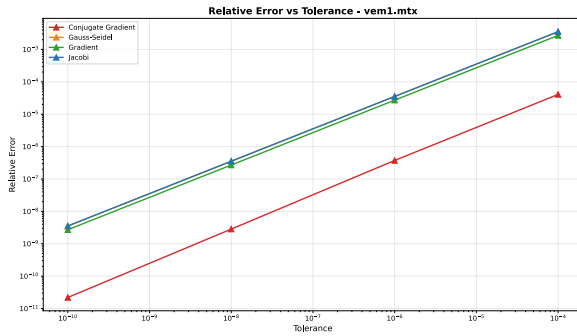
3.2.3 Errore vs Tolleranza



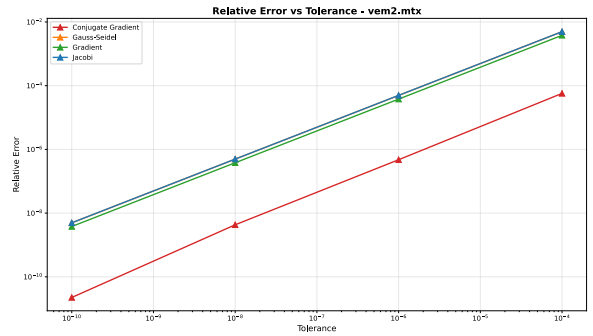
(a) spa1



(b) spa2



(c) vem1



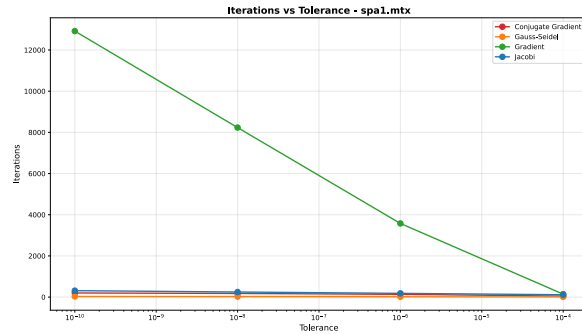
(d) vem2

L'errore relativo finale cresce, così come atteso, con l'aumentare della tolleranza. Il Gradiente Coniugato tende spesso a produrre errori finali inferiori di un ordine di grandezza rispetto alla tolleranza fissata (ad esempio su **vem1** per 10^{-10} restituisce un errore di $2.19 \cdot 10^{-11}$). Le differenze di accuratezza tra i metodi a parità di tolleranza sono comunque trascurabili ai fini pratici. La scelta del risolutore deve quindi ricadere unicamente su considerazioni di efficienza.

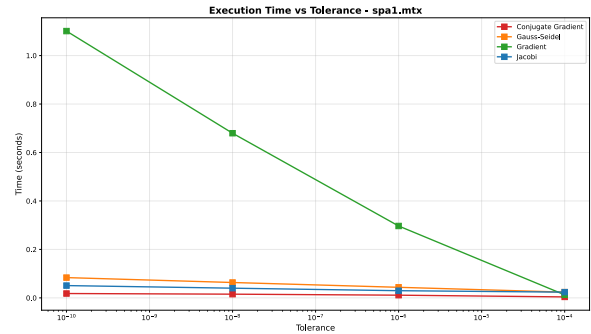
Si noti che, a parità di Tolleranza, i metodi di Jacobi e Gauss-Seidel producono un errore pressoché identico, per questo non è apprezzabile alcuna differenza osservando i grafici sopra riportati.

3.2.4 Confronto per Famiglia di Matrici

Famiglia spa



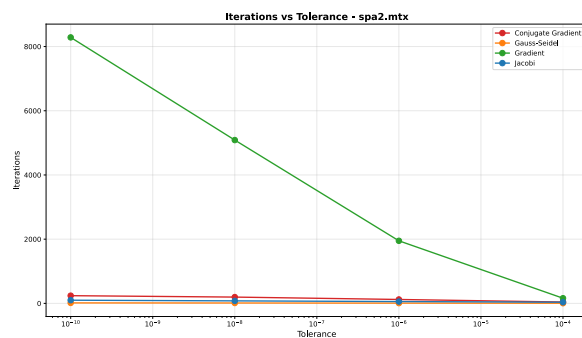
(a) Iterazioni



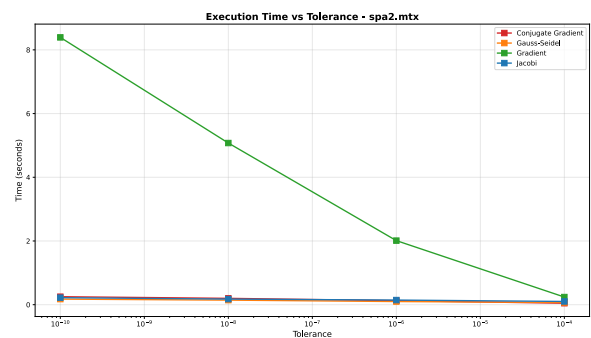
(b) Tempo



(c) Errore relativo



(a) Iterazioni



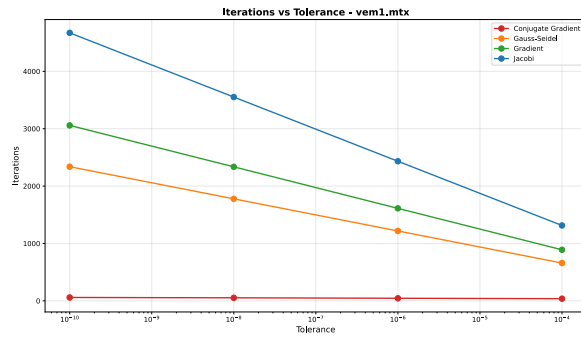
(b) Tempo



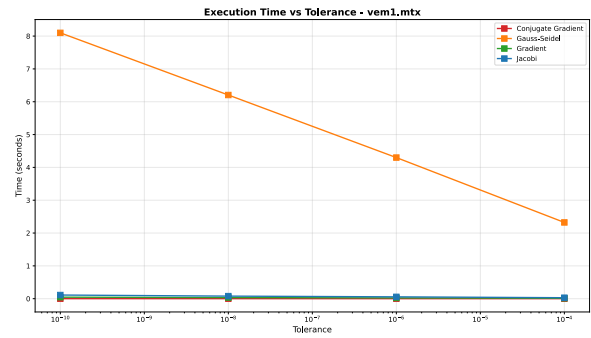
(c) Errore relativo

Sulle matrici della famiglia `spa`, metodi stazionari come Gauss-Seidel sfruttano le proprietà della matrice, risultando non solo competitivi, ma persino superiori in vari scenari di tolleranza sia per iterazioni che per tempo (come su `spa2`). Nonostante ciò, il Gradiente Coniugato garantisce un costo temporale ridotto (spesso sotto il secondo), mantenendosi comunque una scelta valida.

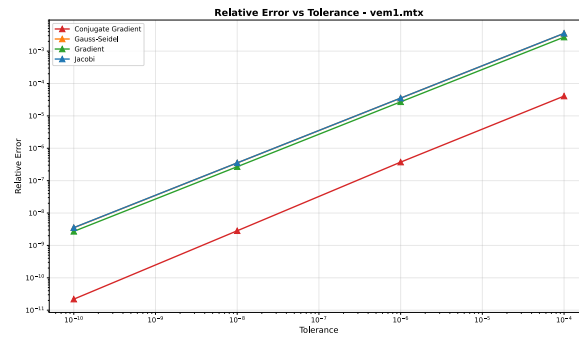
Famiglia vem



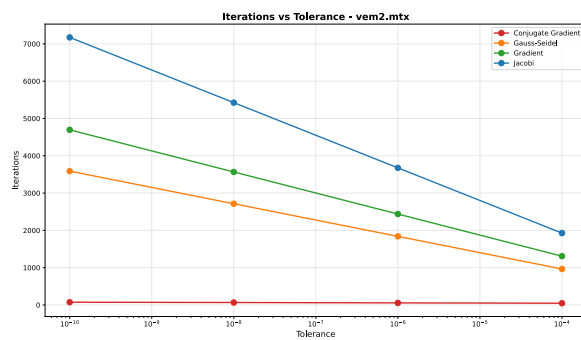
(a) Iterazioni



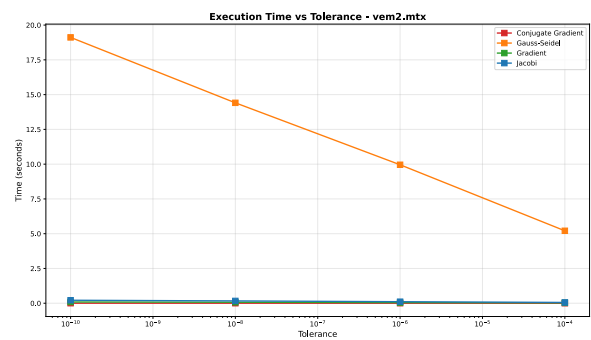
(b) Tempo



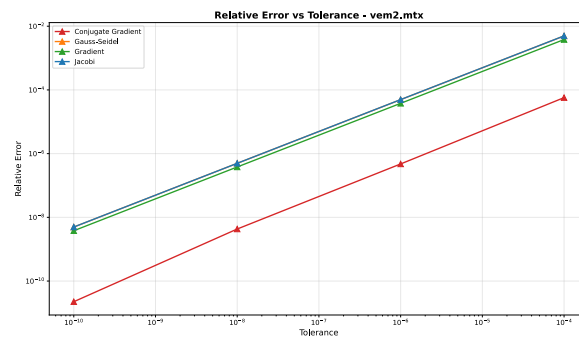
(c) Errore relativo



(a) Iterazioni



(b) Tempo



(c) Errore relativo

Sulle matrici della famiglia **vem**, si notano i punti deboli dei metodi stazionari classici. I tempi di calcolo per Jacobi e Gauss-Seidel diventano impegnativi a causa di un elevatissimo numero di iterazioni. Questo comportamento è strettamente legato al numero di condizionamento $\kappa(A)$ o al raggio spettrale delle matrici: le matrici **vem** risultano peggio condizionate rispetto alle **spa**, rallentando i metodi stazionari. Il condizionamento di una matrice riflette, in termini pratici, la sua stabilità e sensibilità numerica. Le matrici della famiglia **vem** presentano un alto numero di condizionamento che rende difficile per i metodi stazionari classici propagare l'informazione corretta ad ogni passo, costringendoli a migliaia di iterazioni per correggere l'errore. Il Gradiente Coniugato, al contrario, riesce a mitigare questa problematica muovendosi lungo direzioni ottimali senza mai ripassare per la stessa direzione, il che spiega la sua rapidità in questo scenario. Esso risulta quindi essere l'unico risolutore capace di mantenere i tempi estremamente ridotti (nell'ordine dei millesimi di secondo).

3.3 Analisi Complessiva

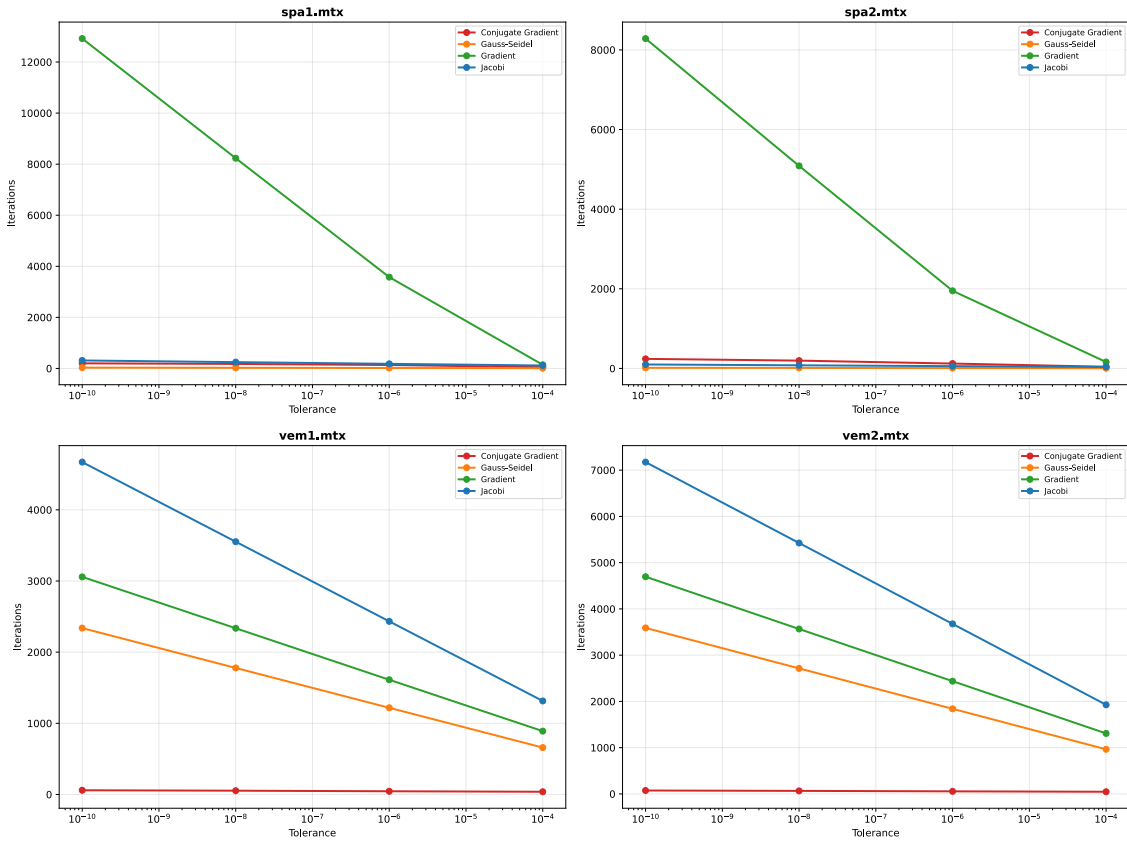


Figura 3.13: Confronto Complessivo

Nel confronto complessivo tra le varie topologie di matrici, le prestazioni dei metodi stazionari risultano altamente variabili. Gauss-Seidel dimostra di poter essere il metodo più rapido in scenari specifici (matrici **spa**), ma peggiora notevolmente su altre tipologie di matrici (**vem**).

Come confermato dalla *Figura 3.13* e dalle tabelle riassuntive, il Gradiente Coniugato offre le performance generalmente migliori: è l'unico metodo in grado di assicurare stabilità e

convergenza in tempi rapidissimi in tutti gli scenari testati, risultando quindi una valida soluzione per applicazioni generiche.

3.3.1 Tabelle Riassuntive

Matrice	Metodo	10 ⁻¹⁰		10 ⁻⁸		10 ⁻⁶		10 ⁻⁴	
		Iter.	Errore	Iter.	Errore	Iter.	Errore	Iter.	Errore
spa1	Conjugate Gradient	200	1.203e-09	177	1.320e-07	134	2.553e-05	49	2.079e-02
	Gauss-Seidel	31	2.248e-08	24	1.710e-06	17	1.300e-04	9	1.821e-02
	Gradient	12919	9.820e-08	8233	9.816e-06	3577	9.680e-04	143	3.457e-02
	Jacobi	313	1.852e-09	247	1.825e-07	181	1.798e-05	115	1.771e-03
spa2	Conjugate Gradient	240	5.324e-09	196	5.587e-07	122	1.198e-04	42	9.821e-03
	Gauss-Seidel	15	5.571e-09	12	2.794e-07	8	5.142e-05	5	2.599e-03
	Gradient	8285	6.938e-08	5087	6.865e-06	1949	6.694e-04	161	1.813e-02
	Jacobi	99	1.484e-09	78	1.573e-07	57	1.667e-05	36	1.766e-03
vem1	Conjugate Gradient	59	2.192e-11	53	2.832e-09	45	3.732e-07	38	4.083e-05
	Gauss-Seidel	2338	3.508e-09	1778	3.517e-07	1218	3.527e-05	659	3.507e-03
	Gradient	3058	2.713e-09	2336	2.695e-07	1612	2.713e-05	890	2.705e-03
	Jacobi	4671	3.539e-09	3552	3.540e-07	2433	3.540e-05	1314	3.540e-03
vem2	Conjugate Gradient	74	2.248e-11	66	4.300e-09	56	4.743e-07	47	5.729e-05
	Gauss-Seidel	3589	4.949e-09	2714	4.958e-07	1840	4.942e-05	965	4.951e-03
	Gradient	4696	3.799e-09	3566	3.810e-07	2438	3.791e-05	1308	3.812e-03
	Jacobi	7174	4.964e-09	5425	4.966e-07	3676	4.967e-05	1927	4.968e-03

Tabella 3.1: Confronto Numero di Iterazioni ed Errore per Tolleranza

Matrice	Metodo	10 ⁻¹⁰ [s]	10 ⁻⁸ [s]	10 ⁻⁶ [s]	10 ⁻⁴ [s]
spa1	Conjugate Gradient	0.017968	0.015314	0.011144	0.003999
	Gauss-Seidel	0.083695	0.063464	0.043632	0.023735
	Gradient	1.101078	0.679539	0.296732	0.012621
	Jacobi	0.050746	0.040012	0.029637	0.023644
spa2	Conjugate Gradient	0.250972	0.199260	0.126159	0.046158
	Gauss-Seidel	0.174983	0.143121	0.098440	0.069385
	Gradient	8.391261	5.074995	2.011299	0.240035
	Jacobi	0.220088	0.175478	0.145432	0.101565
vem1	Conjugate Gradient	0.000995	0.000817	0.000805	0.000651
	Gauss-Seidel	8.098838	6.204676	4.299073	2.323009
	Gradient	0.045936	0.034216	0.023813	0.013934
	Jacobi	0.111441	0.078420	0.056335	0.029852
vem2	Conjugate Gradient	0.001771	0.001493	0.001158	0.000987
	Gauss-Seidel	19.114741	14.410115	9.956529	5.209821
	Gradient	0.090640	0.071065	0.045706	0.028627
	Jacobi	0.214703	0.166684	0.110131	0.057225

Tabella 3.2: Confronto Tempi di Esecuzione per Tolleranza

3.4 Processo di Ottimizzazione

Al fine di ottenere i valori riportati nelle tabelle, è stato attuato un processo di analisi del codice, arrivando così ad identificare i principali colli di bottiglia all'interno dei loop, legati principalmente alla gestione della memoria e a calcoli ridondanti.

Le modifiche principali introdotte nella classe baseline Python si dividono nelle seguenti categorie:

- **Estrazione di Costanti e Matrici dal Loop:** Nell'implementazione originale di Jacobi, l'estrazione della matrice ($A - \text{diag}(A)$) veniva ricalcolata a ogni singola iterazione. Nel codice ottimizzato, la derivazione della matrice sparsa `A_offdiag` viene precalcolata una sola volta prima del ciclo. Questo permette di evitare di dover istanziare in memoria una nuova struttura dati CSR sparsa a ogni passo, risparmiando due operazioni di complessità $O(NNZ)$ per ogni loop
- **Operazioni Vettoriali In-Place:** In Python, eseguire operazioni in formato $x = x + \alpha \cdot r$ porta l'interprete ad allocare un nuovo array in memoria ad ogni esecuzione. Sfruttare operatori in-place (`x += alpha * r`) permette di alleggerire il carico di lavoro. Questo ottimizza l'implementazione baseline per il calcolo del Gradiente e del Gradiente Coniugato
- **Gestione del Residuo:** Per valutare la precisione della soluzione, l'algoritmo misura lo scarto tra il risultato ottenuto e quello atteso, calcolando la norma tramite `np.linalg.norm(A @ x - b)`. Metodi come il Gradiente e il Gradiente Coniugato tengono già traccia di questo scarto in automatico all'interno del vettore r . Questo permette di verificare l'accuratezza in modo quasi istantaneo controllando semplicemente `np.linalg.norm(r)`, risparmiando al sistema pesanti moltiplicazioni matriciali a ogni passaggio. Nei metodi iterativi classici come Gauss-Seidel, invece, $A \cdot x - b$ deve essere ricalcolato ad ogni singola iterazione portando così ad un aumento esponenziale del costo computazionale

Nella versione Numba del metodo di Jacobi l'approccio adottato è differente rispetto alla baseline: anziché costruire preventivamente la matrice `A_offdiag`, il kernel JIT calcola direttamente il risultato riga per riga, sfruttando i dati grezzi della matrice già in memoria. In pratica si elimina del tutto la necessità di creare una struttura dati ausiliaria, riducendo il lavoro complessivo e il consumo di memoria.

3.5 Accelerazione con Numba

Per spingere ulteriormente le prestazioni oltre i limiti dell'interprete CPython, è stata introdotta la libreria Numba. Numba è un compilatore Just-In-Time (JIT) basato sull'infrastruttura LLVM (Low Level Virtual Machine), progettato specificamente per tradurre dinamicamente funzioni Python e NumPy in codice macchina nativo ad alte prestazioni.

L'integrazione è avvenuta decorando le funzioni critiche con `@njit(fastmath=True)`. Il flag `nopython=True` (implicito in `@njit`) impone a Numba di operare in modalità "no Python": la libreria analizza il bytecode della funzione, esegue un'inferenza dei tipi di dato delle variabili in ingresso (evitando il dynamic typing di Python) e genera codice macchina specifico per l'architettura della CPU ospite, bypassando totalmente la CPython API. Il flag `fastmath=True` abilita ulteriori ottimizzazioni aritmetiche LLVM (come

il riordino delle operazioni in virgola mobile e l'uso di istruzioni Fused Multiply-Add), che contribuiscono a ridurre ulteriormente i tempi di calcolo sui kernel numericamente intensivi.

La traduzione JIT ha un impatto differente sui vari metodi, dipendente dalle operazioni svolte:

- **Metodi Vettorializzati (Gradiente Coniugato, Gradiente, Jacobi):** Nei calcoli standard, Python utilizza NumPy per richiamare routine compilate in C tramite l'operatore `@`. Sostituendo le chiamate SciPy con routine compilate tramite JIT (ottimizzate per il prodotto tra vettori e matrici sparse), Numba itera direttamente sui tre array lineari del formato CSR (`data`, `indices`, `indptr`). In questo modo, l'algoritmo opera in modo nativo su porzioni di memoria contigue, garantendo un'esecuzione estremamente ottimizzata
- **Metodi Sequenziali (Gauss-Seidel):** Il metodo Gauss-Seidel è quello che trae il massimo vantaggio dalla compilazione JIT. Essendo un algoritmo sequenziale, ogni iterazione dipende strettamente dal risultato precedente, il che lo rende impossibile da vettorializzare, è quindi necessario elaborare i dati nodo per nodo, anziché in blocchi paralleli. In Python puro, scorrere uno a uno gli indici e gli array del formato CSR comporta un enorme costo computazionale, a causa dei continui cicli annidati per leggere l'inizio e la fine di ogni riga. Passando invece a un ciclo `for` nativo compilato in C tramite Numba, questo collo di bottiglia viene rimosso. L'efficienza diventa paragonabile a quella del codice a basso livello, spiegando così il crollo dei tempi di calcolo

Matrice	Conjugate Gradient [s]		Gauss-Seidel [s]		Gradient [s]		Jacobi [s]	
	Base	Numba	Base	Numba	Base	Numba	Base	Numba
spa1	0.018	0.033	0.084	0.011	1.101	2.050	0.051	0.049
spa2	0.251	0.375	0.175	0.053	8.391	12.910	0.220	0.150
vem1	0.001	0.001	8.099	0.081	0.046	0.049	0.111	0.075
vem2	0.002	0.002	19.115	0.195	0.091	0.117	0.215	0.179

Tabella 3.3: Confronto Tempi di Esecuzione Baseline vs Numba (Tolleranza 10^{-10})

Capitolo 4

Conclusioni

Il progetto ha permesso di implementare, ottimizzare e validare una libreria modulare in Python per la risoluzione di sistemi lineari sparsi, simmetrici e definiti positivi, confrontando le prestazioni di quattro metodi iterativi classici: Jacobi, Gauss-Seidel, Gradiente e Gradiente Coniugato.

L'analisi condotta su matrici appartenenti a due diverse famiglie (**spa** e **vem**) e su diverse tolleranze ha confermato le basi teoriche dei metodi, garantendo la convergenza in tutti gli scenari di test. Tuttavia, sono emersi divari sostanziali in termini di efficienza computazionale, scalabilità e impatto delle ottimizzazioni a livello di architettura del codice.

4.1 Valutazione dei Metodi Iterativi

Dall'osservazione del numero di iterazioni e dei tempi di esecuzione, emergono prestazioni nettamente differenti per ciascun algoritmo:

- **Jacobi e Gradiente:** Entrambi i metodi si sono rivelati poco competitivi per la risoluzione efficiente dei sistemi proposti. Hanno richiesto un numero di iterazioni ampiamente superiore alle alternative. Sebbene beneficino della vettorializzazione intrinseca di NumPy, i tempi di calcolo restano elevati (in particolare per le matrici **vem**). Il Gradiente, pur mostrando un numero di iterazioni ridotto rispetto a Jacobi, non è riuscito a pareggiare le prestazioni degli altri metodi, evidenziando un costo computazionale per singolo passo troppo oneroso
- **Gauss-Seidel:** Il metodo ha mostrato un comportamento fortemente dipendente dalla topologia della matrice. Sulla famiglia **spa**, converge molto rapidamente, risultando il metodo più veloce in assoluto (ad esempio su **spa2**). Sulle matrici **vem**, la versione baseline evidenzia gravi problematiche legate all'elevato numero di iterazioni unito alla sua natura strettamente sequenziale. Proprio per questo motivo, è stato l'algoritmo a trarre il massimo vantaggio dall'introduzione della compilazione JIT tramite Numba, passando da tempi di decine di secondi a frazioni di secondo
- **Gradiente Coniugato:** È risultato essere l'algoritmo più robusto, efficiente e scalabile dell'intera libreria. Oltre a garantire un numero di iterazioni controllato e indipendente dal condizionamento delle matrici **vem**, ha un costo computazionale per passo particolarmente ridotto. Ciò gli ha permesso di ottenere ottimi risultati

nelle matrici `vem` (dove risolve il sistema nell'ordine dei millesimi di secondo) e altamente competitivo sulla famiglia `spa`.

4.2 Impatto dell'Ottimizzazione Numerica e del JIT

Il lavoro ha evidenziato come l'ottimizzazione del codice sia un fattore rilevante, in particolar modo in linguaggi come Python. Un'attenta gestione della memoria, come la pre-allocazione dei buffer, l'impiego di calcoli vettoriali in-place e l'estrazione di matrici costanti dai cicli iterativi, ha permesso di ottenere miglioramenti significativi a livello di *baseline*.

L'introduzione della compilazione Just-In-Time in supporto alle istruzioni native Python ha offerto diversi spunti di riflessione. L'efficacia di Numba si è rivelata essere dipendente dalla struttura di ciascun algoritmo e non universale:

- **Metodi Sequenziali:** Per Gauss-Seidel, che in Python soffre dell'enorme costo di esecuzione dei cicli `for`, la traduzione in linguaggio macchina ha portato a miglioramenti importanti, abbattendo i tempi di quasi due ordini di grandezza (es. da $\sim 19s$ a $\sim 0.19s$ su `vem2`)
- **Metodi Vettorializzati:** Per algoritmi come il Gradiente Coniugato, il cui carico computazionale (prodotto matrice-vettore sparso) è delegato a routine di libreria di bassissimo livello (come BLAS/LAPACK), scritte in C o Fortran e richiamate nativamente da SciPy. In questo scenario, il compilatore JIT di Numba non ha margine per migliorare ulteriormente queste operazioni già ottimizzate per l'hardware. Anzi, il tentativo di traduzione introduce un overhead che porta a leggeri rallentamenti rilevati sulle matrici `spa`.